



GAMEWATCH

PLAY • MANAGE • CONNECT

The All-in-One PS Management System
for Staff, Players & Customers



PLAY
FOR PLAYERS



MANAGE
FOR STAFF



CONNECT
FOR CUSTOMERS

System Architecture & Build Plan

Flutter · FastAPI · PostgreSQL · Edge AI

One app. One cloud backend. One separate camera/AI engine.

From a working testing release in ~3 days
to a one-week pilot and a hardened release candidate.

Contents

The technical blueprint behind the GameWatch pilot.

Part I — Architecture	4
1 The Architecture Decision	4
2 Full System Architecture	5
3 Why the AI Stays at the Edge	6
4 GameWatch Edge Agent	6
Part II — Platform	7
5 Cloud, Database & Storage	7
6 Auth, Roles & Security	8
7 The Flutter App	9
8 API, Realtime, Ads & Subscriptions	10
Part III — Delivery	11
9 Deployment & Environments	11
10 Testing Strategy	13
11 Camera & Security Testing	14
Part IV — The Pilot	15
12 The Three-Day Build Plan	15
13 Pilot Week & MVP Scope	15
14 The Locked Build Order	16

Architecture at a Glance

Do NOT build separate Android, tablet, and web applications. Build one Flutter application with responsive layouts, backed by one cloud backend, while keeping the camera/AI engine as a separate Python service.

CODEBASE 1 Flutter	TARGETS 3 phone/tablet/web	SYSTEMS 4 app/API/DB/edge	BUILD 3 days testing release	PILOT 7 days hardening
---------------------------------	---	--	---	-------------------------------------

The stack in five layers

FLUTTER APP	One codebase: Android phone, Android tablet and web — owner, clerk and customer experiences
FASTAPI BACKEND	Auth, permissions, business logic, payments, games, shifts, tournaments, reports — the brain of the business system
SUPABASE CLOUD	PostgreSQL database, Auth, file Storage and Realtime — infrastructure we don't rebuild in three days
GAMEWATCH EDGE	Python / OpenCV on the lounge PC: preprocessing, scoreboard reader, time-aware brain, match verifier, local event queue
CAMERAS	CCTV, webcam and phone cameras watching the TVs

The Architecture Decision

Four systems, each with one job

Sections 1-2

A · GAMEWATCH APP

Flutter

Runs on:

- ✓ Android phone
- ✓ Android tablet
- ✓ Web browser

B · BACKEND

FastAPI

Python — responsible for:

- ✓ users & authentication
- ✓ lounges, clerks, TVs, cameras
- ✓ games, payments, shifts
- ✓ tournaments, promotions
- ✓ subscriptions & APIs

C · DATABASE

PostgreSQL

Source of business truth

- ✓ structured business data
- ✓ via Supabase

D · EDGE / AI

Python

OpenCV at the lounge

- ✓ runs where the cameras are
- ✓ heavy camera AI stays here

The app does not directly run your heavy camera AI.

USER-FACING

Flutter

+

BUSINESS BRAIN

FastAPI

+

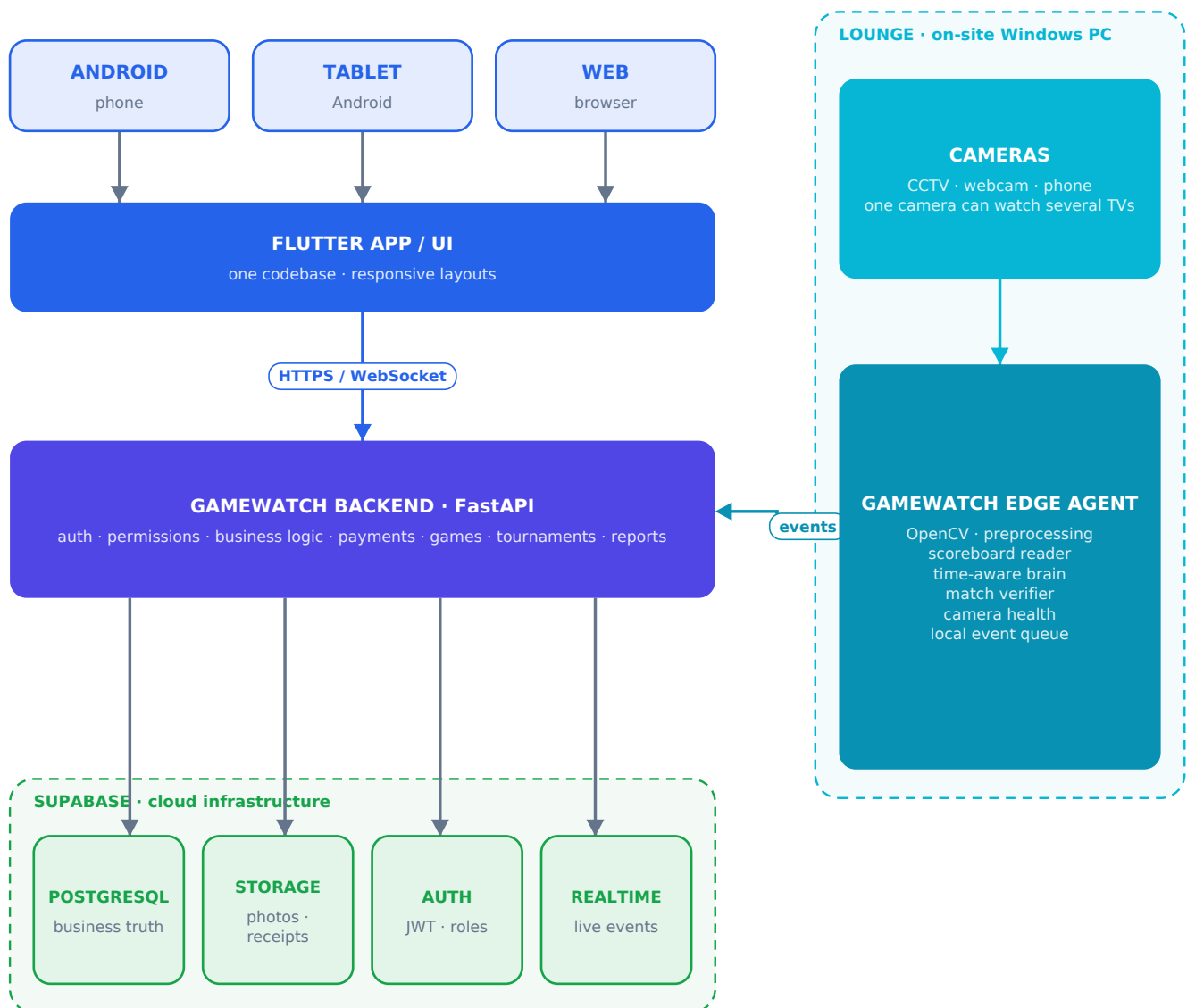
BUSINESS TRUTH

PostgreSQL

+

CAMERA / AI

Edge



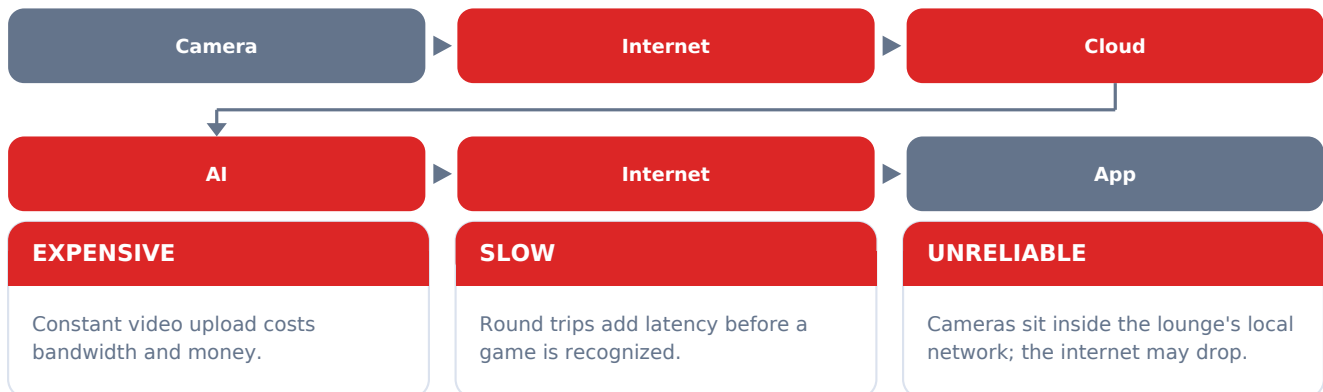
The edge agent sends structured events — never continuous video — to the backend. The app only talks to the backend.

Flutter = everything users touch.

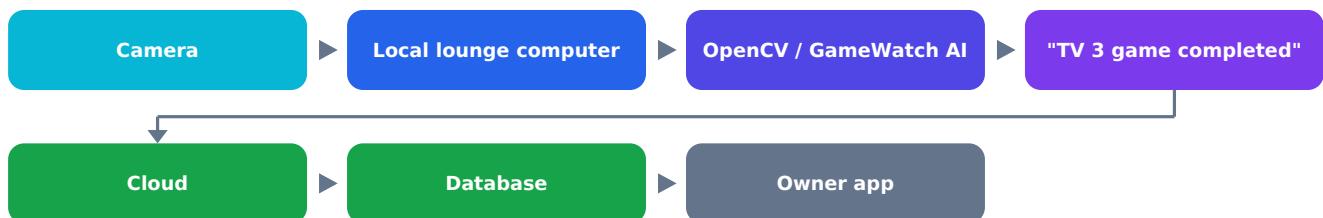
FastAPI = the brain of the business system.

PostgreSQL = source of business truth.

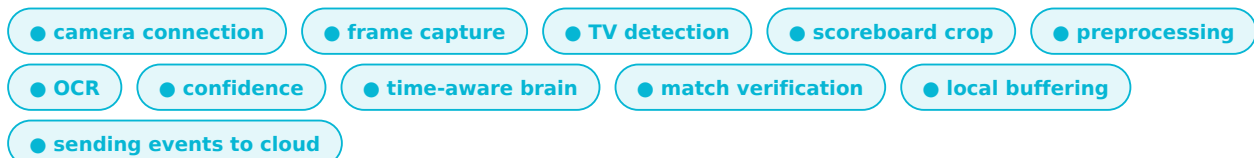
Not this — every frame travels twice



This — process locally, send only the result



Much lighter: cameras and heavy AI stay on the lounge network; only a small event reaches the cloud.



```

event.json · what the edge sends

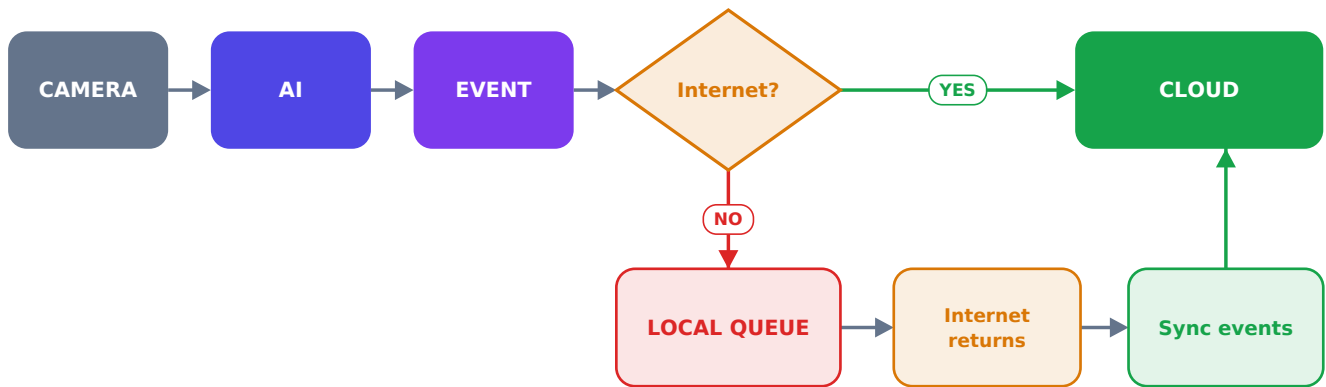
{
  "lounge_id": "lounge_123",
  "tv_id": "tv_3",
  "event": "MATCH_COMPLETED",
  "clock": "90:00",
  "confidence": 0.94,
  "timestamp": "2026-10-03T18:42:12Z"
}
  
```

BACKEND RECORDS

TV	TV 3
Event	Game completed
Time	18:42
Price	25 ETB

It should not continuously upload video — only structured events.

What if the internet goes down? — local event queue



The lounge doesn't lose game records just because its internet temporarily disappears. Build this from the beginning.

EDGE MAY ONLY

- ✓ send detection events
- ✓ send health status
- ✓ request configuration

EDGE MAY NOT

- ✗ read owner financial information
- ✗ modify subscriptions
- ✗ create staff
- ✗ change prices

PART II — PLATFORM

5

Cloud, Database & Storage

Supabase gives us four building blocks on day one

Sections 6–8

POSTGRESQL

Full database for structured business data.

AUTH

Authentication with JWT-based authorization.

STORAGE

Object storage for images and documents.

REALTIME

Live events pushed to the app.

Saves building an authentication server, database server, file server and realtime infrastructure from scratch.

Database tables



Do NOT put uploaded images inside PostgreSQL. Store files in object storage; keep only the metadata and path in the database.

STORAGE BUCKETS · gamewatch-storage/

Folder layout:

- ▶ payment-proofs/
- ▶ problem-photos/
- ▶ deposit-receipts/
- ▶ promotion-images/
- ▶ lounge-logos/
- ▶ tournament-images/
- ▶ camera-diagnostics/

DATABASE REFERENCE

PostgreSQL stores a pointer, not the image:

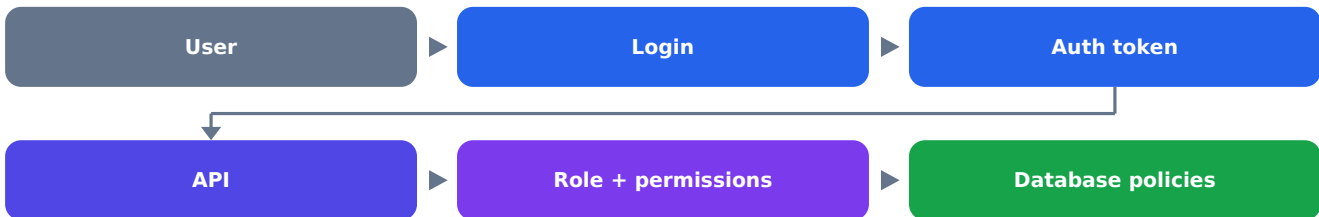
payment_proof_id	uuid
transaction_id	uuid
storage_path	payment-proofs/...
uploaded_by	user
uploaded_at	timestamp

6

Auth, Roles & Security

The backend is the authority — never the frontend

Sections 9–10, 22–24



Use proper authentication (Supabase Auth, JWT + Row Level Security) — don't invent our own password system.

Role ≠ permission. Don't write "if role == owner" everywhere — define permissions.

OWNER

- ✓ view_dashboard
- ✓ manage_staff
- ✓ manage_tvs
- ✓ manage_cameras
- ✓ view_revenue
- ✓ manage_tournaments
- ✓ manage_subscription

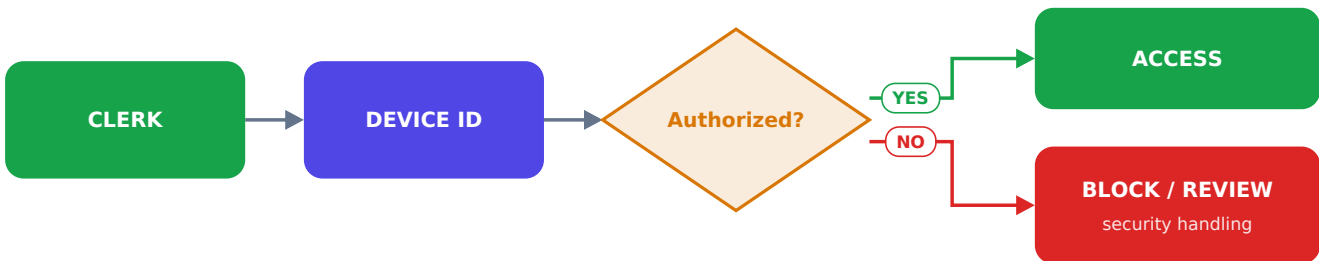
CLERK

- ✓ start_shift
- ✓ manage_games
- ✓ record_payment
- ✓ report_problem
- ✓ end_shift

CUSTOMER

- ✓ view_lounge
- ✓ view_availability
- ✓ join_tournament
- ✓ view_results

Clerk device authorization



Never trust the frontend

NOT ENOUGH

```
if user.isOwner: showRevenue()
```

Someone could modify the app and manipulate the client.

THE RIGHT FLOW

Every request is checked on the server:

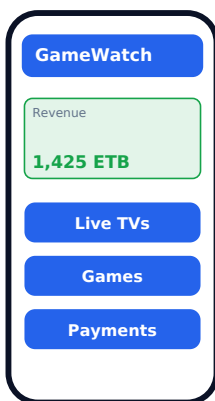


7

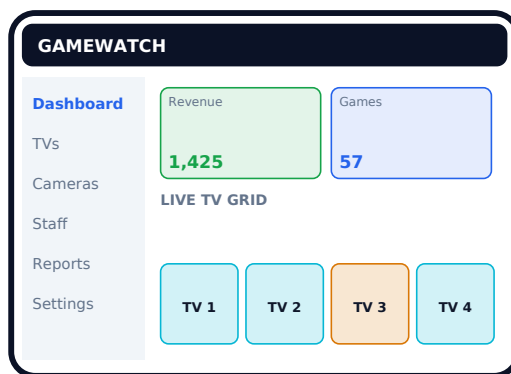
The Flutter App

One codebase, responsive layouts — not a stretched phone UI

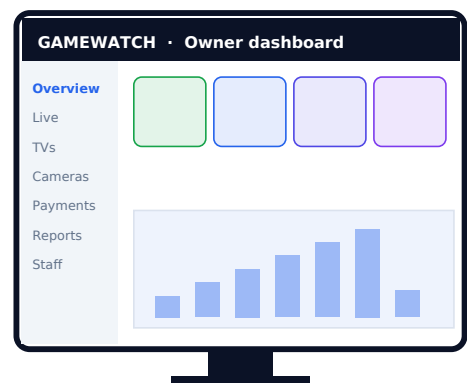
Sections 11-15



PHONE



TABLET



DESKTOP WEB

OWNER · WEB / TABLET

Full dashboard on a PC:

- ▶ Overview
- ▶ Live Lounge
- ▶ TVs
- ▶ Cameras
- ▶ Games
- ▶ Payments
- ▶ Staff
- ▶ Reports
- ▶ Intelligence
- ▶ Tournaments
- ▶ Promotions
- ▶ Customers
- ▶ Subscription
- ▶ Settings

CLERK · PHONE / TABLET

Extremely fast — no 7 taps to record a game:

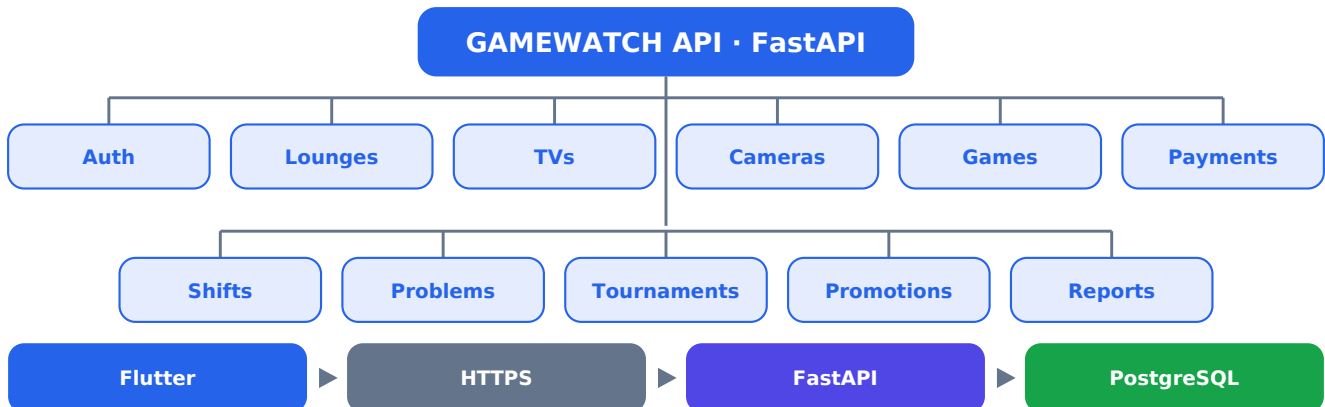
- ▶ Today
- ▶ Live TVs
- ▶ Games
- ▶ Payments
- ▶ Problems
- ▶ My Shift

CUSTOMER · ANDROID / WEB

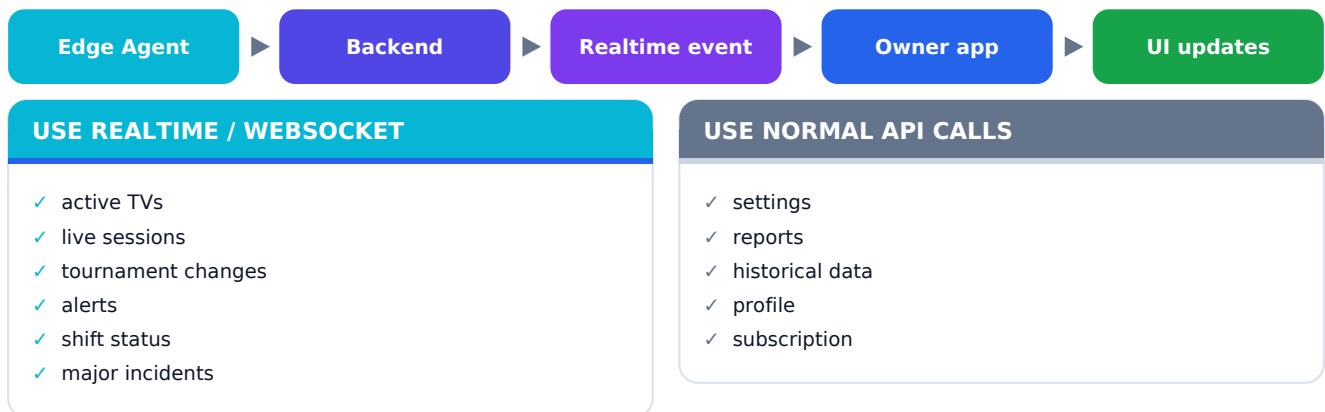
Same app, customer experience:

- ▶ Discover
- ▶ My Lounges
- ▶ Available TVs
- ▶ Happy Hour
- ▶ Tournaments
- ▶ Promotions
- ▶ Profile

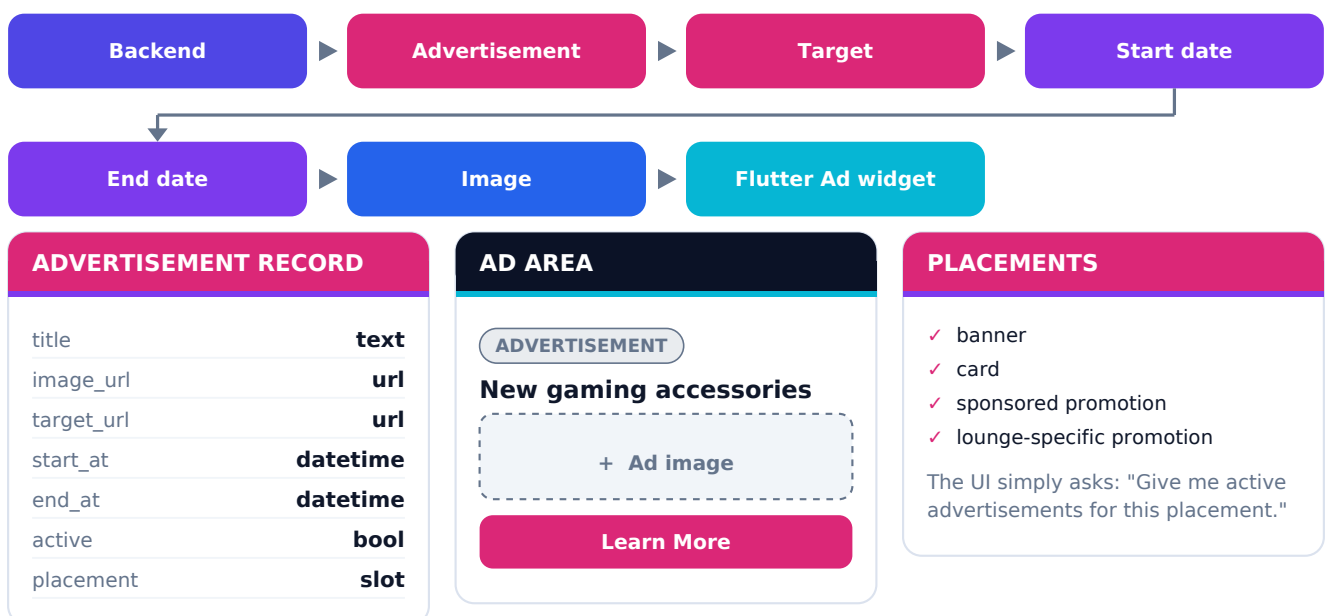
API architecture



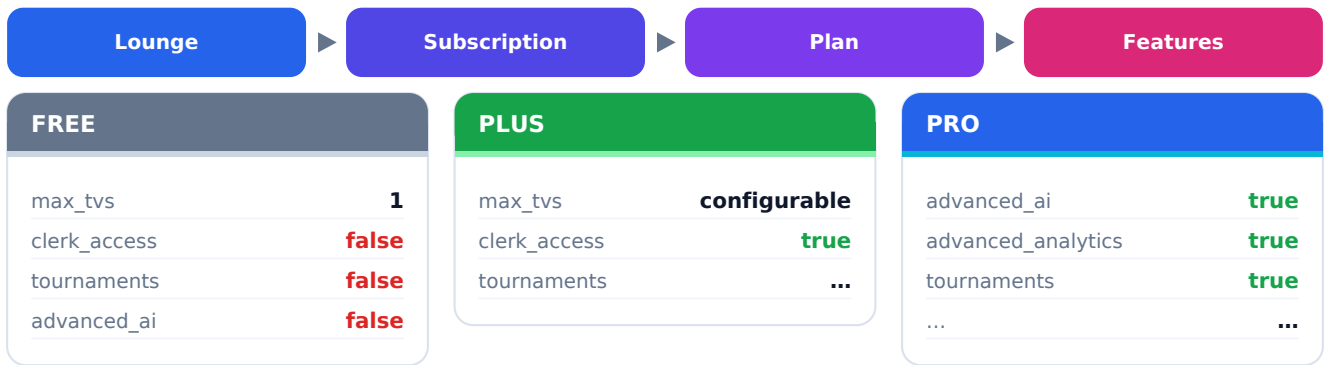
Realtime — only where it earns its place



Ads are data, not hardcoded UI



Subscriptions are enforced by the backend



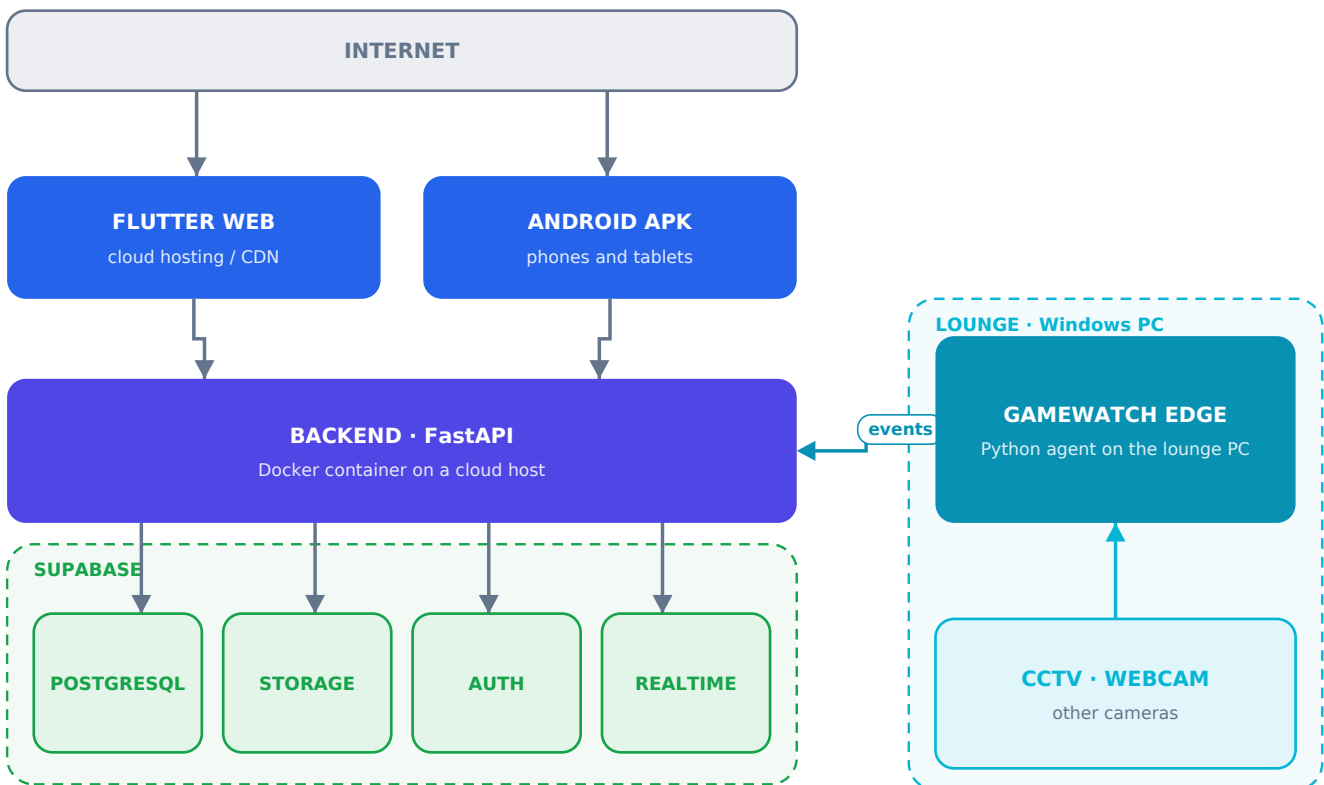
The UI should not decide whether someone may use a feature. The backend must enforce it.

PART III — DELIVERY

9 Deployment & Environments

Where each piece runs

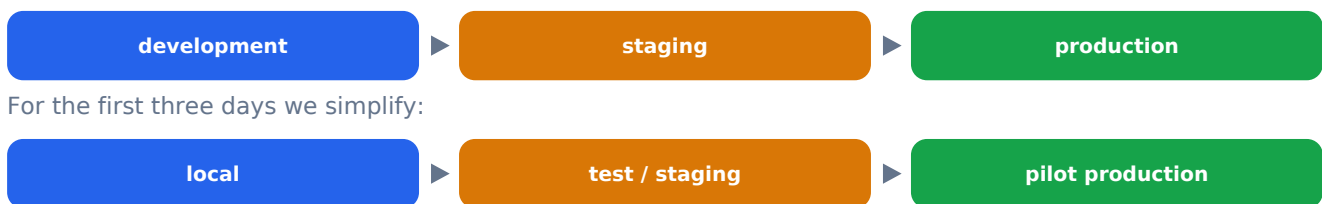
Sections 25–29



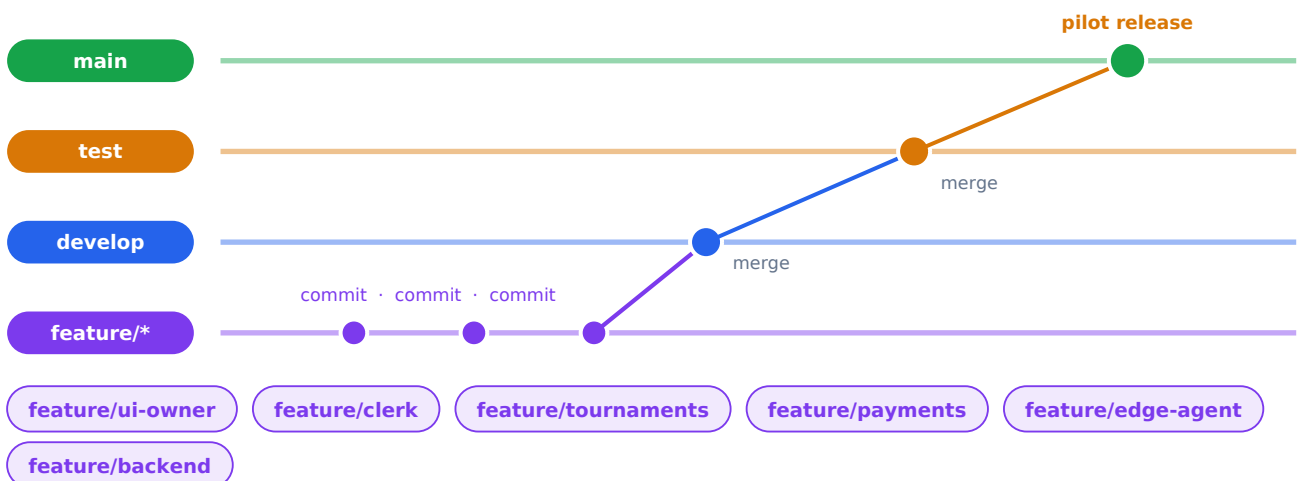
Where we deploy each part

● Flutter Web	Cloud hosting / CDN
● FastAPI	Docker container on a cloud host
● Database	Supabase PostgreSQL
● Storage	Supabase Storage
● Authentication	Supabase Auth
● AI Edge	Lounge computer
● Source code	GitHub

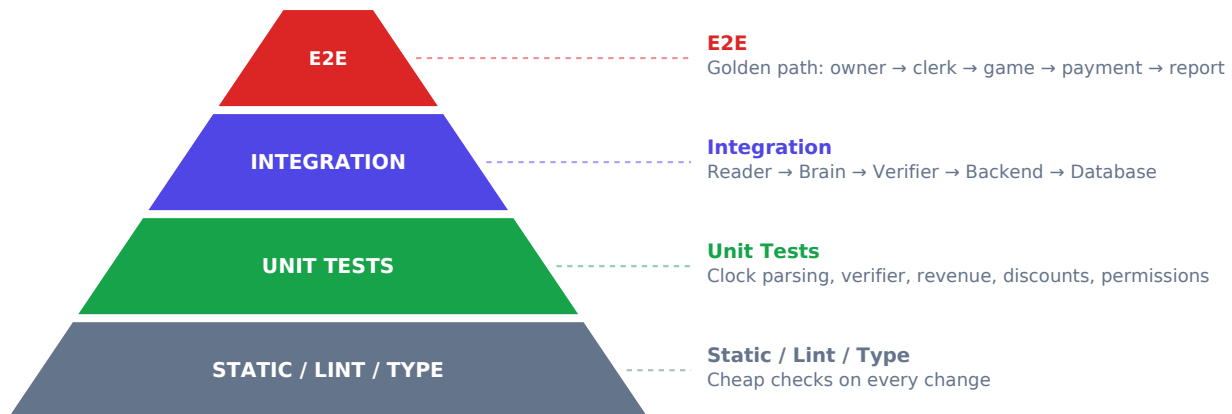
Environments



Git branching



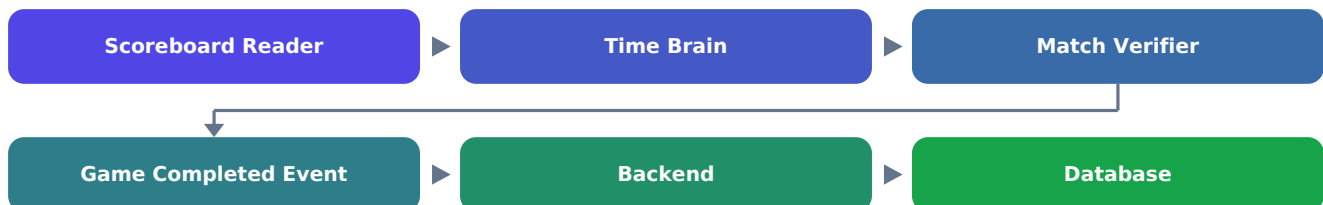
Don't create 30 branches just because we can. First release: main, develop, test, feature/*.



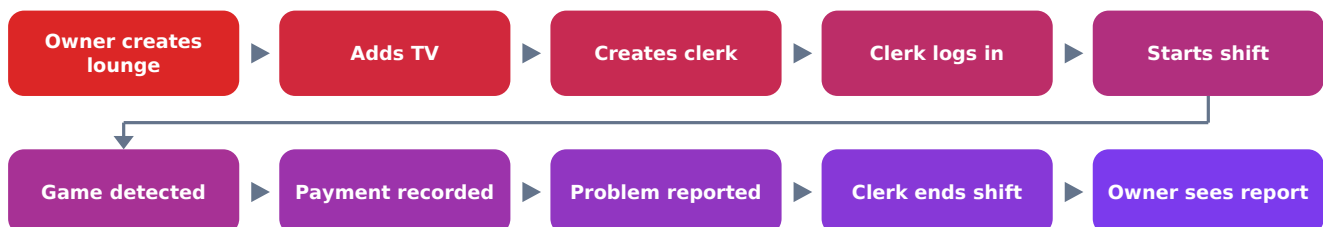
Unit tests — examples

MATCH CLOCK	GAME VERIFICATION	REVENUE & DISCOUNT
00:51 51 s	kickoff progress	20 × 25 500 ETB
01:21 81 s	finish	500 – 100 400 ETB
03:15 195 s	Only a verified kickoff → progress → finish counts as a game.	
99:99 REJECT		

Integration test — the whole chain



E2E — the golden test



11 Camera & Security Testing

Try to break it before the lounge does

Sections 34, 35, 38

NORMAL

Clock reads advance:

00:51

01:21

03:15

...

90:00

BLOCKED CAMERA

visible

blocked

visible

Failure must be visible,
not silent.

OCR ERROR

40:12 → 80:12

Don't blindly accept
impossible progression.

STARTS LATE

begins at 40:00

Should not automatically
count a new match.

COVERAGE TARGETS · pytest

Unit tests 80%+

Critical business logic 90%+

Coverage % alone isn't quality — critical areas
get explicit scenario tests.

CRITICAL CODE

- ✓ match verifier
- ✓ time-aware brain
- ✓ game counter
- ✓ payments
- ✓ discounts
- ✓ authorization
- ✓ subscription limits
- ✓ shift calculations
- ✓ tournament bracket
- ✓ revenue calculations

phone width

tablet width

desktop width

Flutter widget + integration tests: login, owner dashboard, clerk shift, payment, problem report, tournament, responsive layouts.

Security testing — before the pilot

ACTOR	ATTEMPT	EXPECTED
Clerk	attempts to access owner revenue	MUST FAIL
Customer	attempts to access private lounge records	MUST FAIL
Clerk	attempts to modify subscription	MUST FAIL
Unauthorized device	attempts to login	TRIGGERS SECURITY HANDLING
User	attempts to change a transaction without permission	MUST FAIL

The Three-Day Build Plan

Finish the core pilot product — not every feature in the PRD

Section 39

DAY 1 · FOUNDATION

Backend

FastAPI · PostgreSQL/Supabase ·
Auth · Lounge · User · TV · Camera ·
Game · Session · Transaction · Shift ·
Problem

Flutter

Login · role selection · owner
dashboard · clerk dashboard ·
responsive navigation

AI

Reader → time-aware brain → match
verifier → game event

Deployment

GitHub · Supabase · backend hosting
· Flutter web deployment

DAY 2 · CONNECT

AI → Backend → Database → Flutter

Owner

- ✓ live TVs
- ✓ game counts
- ✓ revenue
- ✓ staff
- ✓ cameras

Clerk

- ✓ login
- ✓ device
- ✓ start shift
- ✓ games
- ✓ payments
- ✓ problems
- ✓ end shift

Evidence

- ✓ payment photos
- ✓ problem photos

DAY 3 · HARDENING

No major new features. We test.

- ✓ Detection: 50+ scenarios
- ✓ Backend: API tests
- ✓ Security: permission attacks
- ✓ Flutter: phone / tablet / web
- ✓ Database: create → read → update → delete
- ✓ Payments: cash / digital / adjustments
- ✓ Shift: start → operate → end → next day
- ✓ Deployment: fresh installation
- ✓ Failures: internet, camera, backend down, invalid login, unauthorized device

Day 2 data path

AI / Edge

Backend

Database

Flutter

Pilot Week & MVP Scope

Don't call it production yet — run it for a week

Sections 40–42

Week-long pilot

Day 1
Internal testing

Day 2
Controlled lounge testing

Day 3
Real camera testing

Day 4
Stress testing

Day 5
Security testing

Day 6
Fixes

Day 7
Release candidate

● detection failures

● false game counts

● missed games

● camera problems

● payment issues

● clerk mistakes

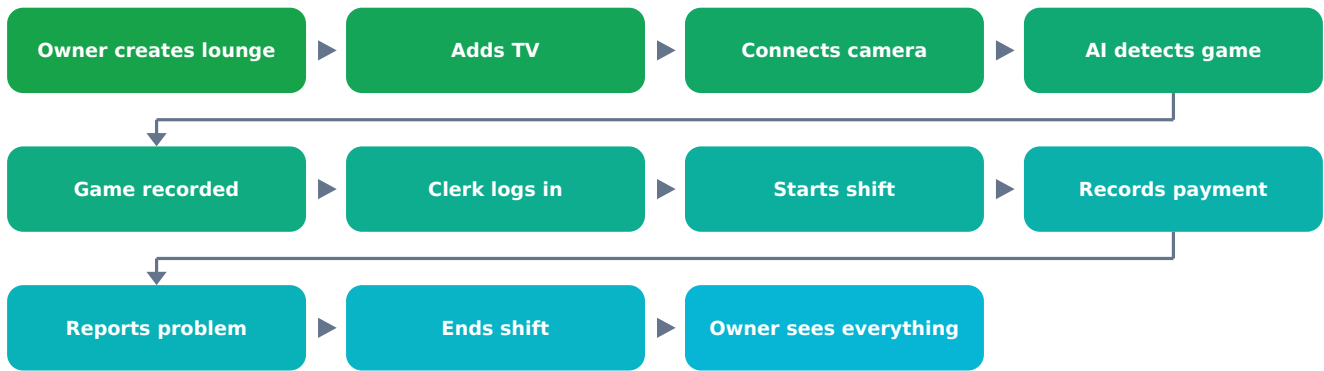
● UI confusion

● network failures

● database problems

Collected during the week.

First release MVP — prove this path



If that works reliably, we have a product.

POSTPONE (ARCHITECTURE SUPPORTS THEM)

- sophisticated customer discovery
- full tournament ecosystem
- advanced loyalty system
- sophisticated ad marketplace
- advanced AI recommendations
- automated Happy Hour recommendations
- international payment integrations
- complicated subscription billing
- advanced customer analytics

THEN LAYER ON TOP

- + Happy Hour
- + Customers
- + Promotions
- + Tournaments
- + AI business intelligence
- + Subscriptions
- + Ads

14

The Locked Build Order

A real architecture, not a prototype where UI, AI and database are tangled

Section 43 + conclusion

FLUTTER

Everything users touch.

FASTAPI

The brain of the business system.

POSTGRESQL

Source of business truth.

STORAGE

Images, receipts and photos.

GAMEWATCH EDGE

Camera and AI processing.

ANTIGRAVITY

Optional intelligence / diagnostic layer.

SUPABASE

Database + auth + storage + realtime, without rebuilding infrastructure.

Build order

